KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

# VS Code에서 Jupyter 노트북 쓰기

셀 실행하는 법과
자주 쓰는 단축키

| | |
|---|---|
| Shift + Enter | 실행하고 다음 셀로 |
| Esc | 명령 모드 |
| B | 아래에 셀 추가 |
| M | 마크다운 셀로 |

# 오늘의 순서


1

**노트북 화면 읽기**

커널, 버튼, 실행 번호, 코드와 마크다운 셀

4–5쪽

2

**셀 실행하기**

세 가지 실행 키, 실행 순서, Restart, 위쪽 버튼

6–12쪽

3

**명령 모드와 편집 모드**

Esc와 Enter, 키로만 셀 만들기, 글자 키

13–18쪽

4

**노트북에 수업 내용 정리하기**

변수 이름 규칙, 컨벤션, 자료형과 type()

19–23쪽

5

**단축키 모음**

Windows와 Mac 나란히

24–25쪽


3쪽은 09에서 한 준비, 26–27쪽은 막힐 때와 한 장 요약이에요.

들어가기

# 준비는 09에서 마쳤어요

- ✓ **Python, Jupyter 확장 설치** — 09 · 25쪽
- ✓ **`uv add --dev ipykernel`** — 09 · 26쪽
- ✓ **커널로 .venv 고르기** — 09 · 27쪽

셋 중 하나라도 안 했으면 09 슬라이드의 그 쪽부터 따라 해요.

### 새 노트북 만들기

1. 명령 팔레트 열기
   - Windows: `Ctrl` + `Shift` + `P`
   - Mac: `Cmd` + `Shift` + `P`
2. `Create: New Jupyter Notebook`
3. 저장하고 오른쪽 위 커널이 .venv인지 보기

**이번 챕터는 09에서 만든 hello-uv 폴더를 VS Code로 열고 시작해요.**

1. 노트북 화면 읽기

# 노트북 화면은 다섯 곳만 알면 돼요




1 **커널**
셀을 실행하는 파이썬. .venv인지 봐요

2 **위쪽 버튼**
셀 추가, 전체 실행, 다시 시작

3 **선택 막대**
지금 고른 셀을 표시해요

4 **실행 번호**
몇 번째로 실행했는지

5 **출력**
실행 결과가 셀 바로 아래에

**셀은 칸이에요. 칸마다 따로 실행하고, 결과는 그 칸 아래에 남아요.**

1. 노트북 화면 읽기

# 셀은 코드 셀과 마크다운 셀 두 가지예요

## 코드 셀 · 파이썬을 실행해요

[3]
```python
name = "kim"
name
```

```
'kim'
```

오른쪽 아래에 Python이라고 적혀요.
마지막 줄이 값이면 print 없이도 보여 줘요.

## 마크다운 셀 · 설명 글을 적어요

```markdown
# 오늘 배운 것
- 셀은 따로 실행해요
- **Shift+Enter**로 실행
```



### 오늘 배운 것

- 셀은 따로 실행해요
- **Shift+Enter**로 실행

실행하면 꾸며진 모습으로 바뀌어요

**마크다운 기호는 01에서 본 것과 같아요. 셀 종류는 오른쪽 아래 글자로 구분해요.**

2. 셀 실행하기

# 실행하고 그 셀에 머물러요

Windows `Ctrl` + `Enter`

Mac `control` + `Enter`

같은 셀을 고치고 다시 돌려 볼 때 써요.

누르기 전

```
[ ]  x = 10                    Python
[ ]  print(x * 2)              Python
```



누른 뒤

```
[1]  x = 10                    Python
[ ]  print(x * 2)              Python
```

**파란 막대가 1번 셀에 그대로 있어요.**

2. 셀 실행하기

# 실행하고 다음 셀로 넘어가요

| | | | | |
|---|---|---|---|---|
| Windows | Shift | + | Enter | 위에서 아래로 차례대로 실행할 때 써요. 가장 많이 써요. |
| Mac | Shift | + | Enter | |

누르기 전

```
[ ]  x = 10                    Python
[ ]  print(x * 2)              Python
```



누른 뒤

```
[1]  x = 10                    Python
[ ]  print(x * 2)              Python
```

**파란 막대가 2번 셀로 내려갔어요.**

2. 셀 실행하기

# 실행하고 바로 아래에 새 셀을 만들어요

| | | | | |
|---|---|---|---|---|
| Windows | Alt | + | Enter | 실행 결과를 보고 이어서 새 코드를 적을 때 써요. |
| Mac | option | + | Enter | |

누르기 전

```
[ ]  x = 10                    Python
[ ]  print(x * 2)              Python
```



누른 뒤

```
[1]  x = 10                    Python
[ ]                            Python
[ ]  print(x * 2)              Python
```

**새 셀이 끼어들고, 커서가 그 안에서 기다려요.**

2. 셀 실행하기

# 대괄호 숫자는 실행한 순서예요

[1]
```python
x = 10
```

[2]
```python
print(x * 2)
```
```
20
```

[3]
```python
name = "kim"
name
```
```
'kim'
```

[4]
```python
x = x + 1
x
```
```
11
```

**커널이 기억하는 값**

```
x = 11
```

```
name = 'kim'
```

앞 셀에서 만든 값을 뒤 셀이 그대로 써요.

**4번 셀을 한 번 더 실행하면 어떻게 될까요?**

2. 셀 실행하기

# 같은 셀을 다시 실행하면 번호도 값도 바뀌어요

[1]
```
x = 10
```
Python

[2]
```
print(x * 2)
```
Python

```
20
```

[3]
```
name = "kim"
name
```
Python

```
'kim'
```

[5]
```
x = x + 1
x
```
Python

```
12
```

**커널이 기억하는 값**

```
x = 12
```

```
name = 'kim'
```

[4]가 [5]로, 11이 12로 바뀌었어요. 화면 순서가 아니라 누른 순서대로 실행돼요.

**숫자가 1, 2, 3 차례가 아니면 위에서부터 다시 실행해 봐요.**

2. 셀 실행하기

# Restart는 기억한 값을 모두 지워요

**Restart 전**

```
x = 12
```

```
name = 'kim'
```

↻ Restart



**Restart 뒤**

비어 있어요

**해결**

**▷ Run All**

맨 위 셀부터 차례로 실행해서 x = 10부터 다시 기억해요.

[1]

```
print(x * 2)
```

Python

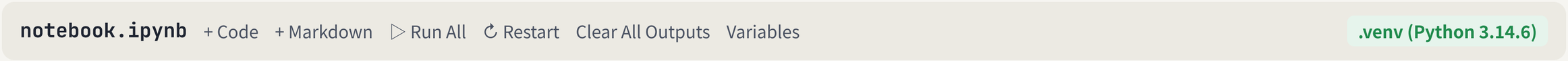
```
NameError: name 'x' is not defined
```

2번 셀만 실행하면 x를 몰라서 오류가 나요. 번호도 [1]부터 다시 세요.

**뭔가 꼬였으면 Restart 뒤 Run All. 위에서부터 깨끗하게 다시 실행해요.**

2. 셀 실행하기

# 위쪽 버튼으로 노트북 전체를 다뤄요




### ▷ Run All

**맨 위부터 끝까지 실행**

처음 열었을 때, Restart 뒤

### ↻ Restart

**커널이 기억한 값 지우기**

결과가 이상하거나 번호가 꼬였을 때

### ☐ Interrupt

**실행 중인 셀 멈추기**

셀이 안 끝날 때. 실행 중에만 보여요

### Clear All Outputs

**출력만 지우기**

값은 그대로 남아요

### Variables

**기억하는 값 표로 보기**

아래쪽에 표가 열려요

**실행 중인 셀**

왼쪽에 도는 표시가 나와요. 오래 걸리면 Interrupt를 눌러요.

3. 명령 모드와 편집 모드

# 셀에는 두 가지 모드가 있어요

## 명령 모드




왼쪽 막대만

셀 자체를 다뤄요
추가, 삭제, 종류 바꾸기

Enter

Esc

## 편집 모드




왼쪽 막대 + 셀 테두리 + 커서

셀 안의 글자를 고쳐요
코드 치기, 지우기

셀 안을 클릭하면 편집 모드, 셀 왼쪽 빈 곳을 클릭하면 명령 모드예요.
Enter와 Esc는 Windows와 Mac이 같아요.

**A, B, M 같은 글자 단축키는 명령 모드에서만 동작해요.**

3. 명령 모드와 편집 모드

# 키로만 셀 만들기 1/4 · 편집 모드에서 Esc

[1]
```
x = 10
```
Python

누른 키

**Esc**

명령 모드가 돼요. 커서와 테두리가 사라져요

**지금은 명령 모드예요. 다음 키는 글자 대신 명령으로 읽혀요.**

3. 명령 모드와 편집 모드

# 키로만 셀 만들기 2/4 · 명령 모드에서 B

[1]
```
x = 10
```
Python

[ ]
```

```
Python

누른 키

B

아래에 새 코드 셀이 생기고 그 셀이 골라져요

**아직 명령 모드라서 M을 치면 셀 종류가 바뀌어요.**

3. 명령 모드와 편집 모드

# 키로만 셀 만들기 3/4 · M을 누르고 Enter, 글 적기

[1]
```
x = 10
```
Python

```
# 오늘 배운 것
```
Markdown

누른 키

M

마크다운 셀로 바뀌어요. Enter로 편집 모드에 들어가 적어요

**편집 모드에서 친 글자는 셀 안으로 들어가요.**

3. 명령 모드와 편집 모드

# 키로만 셀 만들기 4/4 · Shift+Enter

[1]
```
x = 10
```
Python

## 오늘 배운 것

[ ]
Python

누른 키

Shift + Enter

마크다운이 꾸며진 제목으로 바뀌고, 아래에 새 셀이 생겨요

**마우스 없이 설명 칸과 코드 칸을 만들었어요.**

3. 명령 모드와 편집 모드

# 명령 모드에서 쓰는 글자 키

**A**

### 위에 셀 추가

Above

**B**

### 아래에 셀 추가

Below

**D** + **D**

### 셀 지우기

D를 두 번

**Z**

### 지운 셀 되살리기

방금 한 일 되돌리기

**M**

### 마크다운 셀로

Markdown

**Y**

### 코드 셀로

다시 코드로

**L**

### 줄 번호 켜고 끄기

오류 줄 찾을 때

**Enter**

### 편집 모드로

셀 안으로 들어가기

**Windows와 Mac이 같아요. 글자가 셀에 들어가면 편집 모드예요. Esc부터 쳐요.**

4. 노트북에 수업 내용 정리하기

# 마크다운 셀에 규칙을 적어 둬요

적는 모습 (편집 모드)

```
---
### 변수 이름 규칙
- 숫자로 시작 불가
- 공백 불가
- 하이픈(-) 불가
- 예약어 사용 불가
```

Markdown

Shift+Enter 뒤

---

**변수 이름 규칙**

- 숫자로 시작 불가
- 공백 불가
- 하이픈(-) 불가
- 예약어 사용 불가

`---` 가로줄 `###` 작은 제목 `-` 목록

기호 뒤에는 한 칸 띄워요

**수업에서 배운 규칙은 코드 옆 마크다운 셀에 적어 두면 복습할 때 같이 봐요.**

4. 노트북에 수업 내용 정리하기

# 규칙을 어기면 SyntaxError가 나요

### 숫자로 시작

```
1st_name = "kim"
```

```
SyntaxError: invalid decimal literal
```

이렇게 `first_name`

### 공백

```
my name = "kim"
```

```
SyntaxError: invalid syntax
```

이렇게 `my_name`

### 하이픈

```
my-name = "kim"
```

```
SyntaxError: cannot assign to expression here.
```

이렇게 `my_name`

### 예약어

```
class = "kim"
```

```
SyntaxError: invalid syntax
```

이렇게 `class_name`

**띄어쓰기와 하이픈 자리에는 밑줄(_)을 써요. 예약어는 뒤에 단어를 붙여요.**

4. 노트북에 수업 내용 정리하기

# 이름 짓는 표기법은 세 가지예요

```
### 컨벤션
- 스네이크 표기법: user_name
- 파스칼 표기법: UserProfile
- 카멜 표기법: userName
```
Markdown

### 컨벤션

- 스네이크 표기법: `user_name`
- 파스칼 표기법: `UserProfile`
- 카멜 표기법: `userName`

## 스네이크 표기법 snake_case

`user_name`

**파이썬 변수, 함수 이름**

우리는 이걸 써요

## 파스칼 표기법 PascalCase

`UserProfile`

**파이썬 클래스 이름**

단어 첫 글자를 모두 대문자로

## 카멜 표기법 camelCase

`userName`

**자바스크립트에서 많이 써요**

파이썬에서는 잘 안 써요

**변수 이름은 스네이크 표기법으로 user_name처럼 적어요.**

4. 노트북에 수업 내용 정리하기

# 코드 셀에 자료형을 하나씩 적어 봐요

[1]

```python
# 정수형 자료형
age = 25

# 실수 자료형
pi = 3.14

# 불리언 자료형(Boolean)
is_alive = True

# 문자열 자료형
user_name = "kim"

# 리스트 (다른 언어의 배열과 비슷해요)
grade_type = [1, 2, 3, 4, 5]
```

Python

| | |
|---|---|
| `25` | **정수** `int`<br>소수점 없는 숫자 |
| `3.14` | **실수** `float`<br>소수점 있는 숫자 |
| `True` | **불리언** `bool`<br>True 또는 False 두 값만 |
| `"kim"` | **문자열** `str`<br>따옴표로 감싼 글자 |
| `[1, 2, 3, 4, 5]` | **리스트** `list`<br>여러 값을 순서대로. 배열과 비슷해요 |

**# 뒤는 설명이라 실행하지 않아요. 값은 출력 없이 커널이 기억해요.**

4. 노트북에 수업 내용 정리하기

# type()으로 자료형을 물어봐요

[1]
```
age = 25
...
grade_type = [1, 2, 3, 4, 5]
```
Python

[2]
```
print(type(age))
print(type(pi))
print(type(is_alive))
print(type(user_name))
print(type(grade_type))
```
Python

```
<class 'int'>
<class 'float'>
<class 'bool'>
<class 'str'>
<class 'list'>
```

- **정수** 소수점 없는 숫자
- **실수** 소수점 있는 숫자
- **불리언** True 또는 False 두 값만
- **문자열** 따옴표로 감싼 글자
- **리스트** 여러 값을 순서대로

**값을 보고 헷갈리면 type()으로 확인해요. 1번 셀을 먼저 실행해야 해요.**

5. 단축키 모음 (Windows, Mac)

# 실행하고 셀 다루는 키

| 하는 일 | Windows | Mac |
| --- | --- | --- |
| **실행하고 머물기** | `Ctrl` + `Enter` | `control` + `Enter` |
| **실행하고 다음 셀로** | `Shift` + `Enter` | `Shift` + `Enter` |
| **실행하고 아래에 새 셀** | `Alt` + `Enter` | `option` + `Enter` |
| **명령 모드로** 편집 모드에서 | `Esc` | `Esc` |
| **편집 모드로** 명령 모드에서 | `Enter` | `Enter` |
| **셀 위아래로 옮기기** 명령 모드에서 | `Alt` + `↑` | `option` + `↑` |
| **셀 추가, 종류 바꾸기** 명령 모드에서 | `A` `B` `M` `Y` | `A` `B` `M` `Y` |

**Mac의 실행 키는 Cmd가 아니라 control이에요.**

5. 단축키 모음 (Windows, Mac)

# 셀 안에서 글 고치는 키와 파일 키

| 하는 일 | Windows | Mac |
| --- | --- | --- |
| **주석 넣고 빼기** 편집 모드에서 | `Ctrl` + `/` | `Cmd` + `/` |
| **되돌리기** 편집 모드에서 | `Ctrl` + `Z` | `Cmd` + `Z` |
| **들여쓰기 빼기** 편집 모드에서 | `Shift` + `Tab` | `Shift` + `Tab` |
| **노트북에서 찾기** | `Ctrl` + `F` | `Cmd` + `F` |
| **노트북 저장** | `Ctrl` + `S` | `Cmd` + `S` |
| **명령 팔레트** | `Ctrl` + `Shift` + `P` | `Cmd` + `Shift` + `P` |
| **확장 화면 열기** | `Ctrl` + `Shift` + `X` | `Cmd` + `Shift` + `X` |

**이 키들은 Windows의 Ctrl이 Mac에서 Cmd로 바뀌어요.**

정리

# 노트북에서 막히면

| | | |
|---|---|---|
| 1 | **글자를 쳤는데 셀이 생기거나 지워져요**<br>명령 모드였어요 | **셀 안 클릭, Enter** |
| 2 | **A, B, M을 눌러도 아무 일이 없어요**<br>편집 모드거나 한글 입력 상태예요 | **Esc, 한/영 키** |
| 3 | **`NameError: name 'x' is not defined`**<br>앞 셀을 안 돌렸거나 Restart 뒤예요 | **▷ Run All** |
| 4 | **`SyntaxError: invalid syntax`**<br>설명 글을 코드 셀에 적었어요 | **명령 모드에서 M** |
| 5 | **셀이 끝나지 않고 계속 돌아요**<br>오래 걸리거나 끝나지 않는 코드예요 | **□ Interrupt** |
| 6 | **Mac에서 Cmd+Enter로 실행이 안 돼요**<br>Mac 실행 키는 control이에요 | **control+Enter** |

# 한 장 요약

`Shift` + `Enter`

**실행하고 다음 셀로**

`Ctrl` + `Enter`

**실행하고 머물기**

Mac control+Enter

`Esc`

**명령 모드로**

`Enter`

**편집 모드로**

`A` `B`

**위, 아래에 셀 추가**

명령 모드

`M` `Y`

**마크다운, 코드 셀로**

명령 모드

**[숫자]는 실행한 순서예요.** 화면 위아래 순서가 아니에요

**꼬이면 ↻ Restart 뒤 ▷ Run All**

**패키지는** `uv add 패키지` 알림의 Install 대신 터미널에서